SOFTOMATIC
TECHNOLOGY SERVICES

EXAMATE · GENAI + PROJECTS · DAY 5 · TUE 27 OCT

GenAI Session 5
Reliable LLM calls + cost

> Yesterday your app called a model. Today it calls it like a product: answers stream, failures are retried safely, a second model steps in, and every question has a price.

SESSION
90 minutes
12:45 Group A + T · repeated 2:15 Group B + T

YOU LEAVE WITH
14 tests passing
streaming /chat, fallback, GET /usage

HOMEWORK
docs/COST.md
before Thu 29 Oct

TODAY'S JOURNEY

01 Stream the answer
02 Retry only what can succeed
03 Fall back to a second model
04 Count tokens and cost
05 **Log every call, add /usage**

DAY 5 · GENAI + PROJECTS

From one call to a production call

Monday you put the model behind an interface and called it from `POST /chat`. A real product needs more: the user should see words at once, a busy provider must not crash the demo, and you must know what each question costs.

By the end of today you can

01

Stream answers
with server-sent events, so words appear in under a second

02

Retry safely
timeouts, backoff and jitter; never retry a 401

03

Fall back
to a second model when the main one is down

04

Price every question
tokens × price, logged, totalled by GET /usage

The 90-minute plan

0–5	Hook	the provider returns 429 during your demo
5–30	Concepts	streaming, retries, fallback, tokens and cost
30–75	Build	streaming client, fallback, cost log, /chat, /usage
75–85	Tests	ruff check . and pytest -q: 14 passed
85–90	Checkpoint	stream one of YOUR questions; read the cost line

Bring

- Your repo with Monday's files
- docs/QUESTIONS.md (your 10 questions)
- Python 3.11+
- An API key for the provider your trainer names (optional)
- pip install httpx

MEMORY HOOK

Stream, Retry, Fall back, Price, Log: "Some Rains Fall, People Look." Five words, five things a production LLM call does.

MISSED MONDAY? START HERE

Monday in one page

Today builds on six things from GenAI 4. If any of them is missing in your repo, type it in from the appendix at the end of these notes before 12:45. It takes about 15 minutes.



Blue: already in your repo from Monday. Green: what you add today.

FILE	WHAT IT DOES	WHERE
`app/config.py + .env`	All settings from environment variables; the API key lives in .env, never in Git	appendix A1
`app/schemas.py`	Message, ChatRequest (max 50 messages, 20,000 characters), Usage, StructuredAnswer	appendix A2
`app/tokens.py`	estimate_tokens(): about 4 characters per token, for logging only	appendix A1
`app/llm/base.py`	The LLMProvider interface: stream_chat(messages, temperature, result)	appendix A3
`app/llm/fake.py`	FakeProvider: answers with no network and no key, for tests and CI	appendix A4
`POST /chat`	The route calls the interface, never a vendor SDK	rebuilt today, section 11

Quick check: are you ready?

- [ ] `.env` exists and is listed in `.gitignore`
- [ ] `app/llm/` has base.py and fake.py
- [ ] `pytest -q` runs (even if only /health passes)

PAUSE AND PREDICT

Why does Monday's fake provider matter so much today?

Answer: today you test timeouts, 429s and fallbacks. With the fake provider and a mock transport you can do that in a second, with no key, no quota and no internet.

YOUR DEMO, MINUTE ONE

The provider says 429

It is your project demo. The interviewer types a question. Your app spins for 30 seconds and then shows **500 Internal Server Error**. In the logs: `429 Too Many Requests`. Your free-tier model hit its rate limit because your batchmate tested at the same time with the same key.

Free tiers are real but tight

OpenRouter's docs limit free model variants to **20 requests per minute** and **50 per day** for accounts that have bought fewer than 10 credits [S1]. Groq publishes per-model free-plan limits, for example **30 requests per minute** and **1,000 per day** for some models, plus token caps [S2]. These numbers change: always read the provider's own limits page.

What today fixes

The same demo after today: the first words appear at once, a 429 is retried after a short random wait, and if the main model is still down, a second model answers. The cost log shows which model answered and what it cost.

PAUSE AND PREDICT

When a provider returns 429, should your code retry immediately?

Answer: no. Retrying at once makes the overload worse, and if 90 students all retry at the same instant they collide again. Wait, wait longer each time (exponential backoff), and add randomness (jitter) so retries spread out.

JUST FOR FUN

Retrying a 429 instantly is like refreshing the Tatkal booking page every 50 milliseconds. You are not getting the ticket; you are getting rate-limited harder.

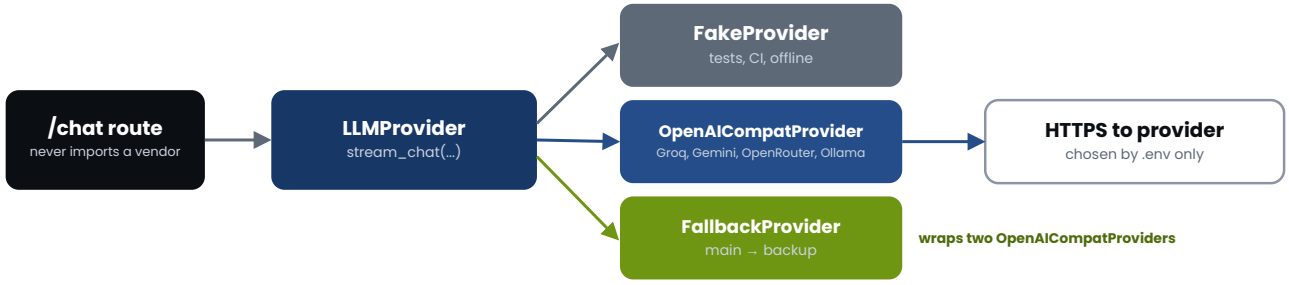
INTERVIEW MODE

"Hosted LLMs enforce rate limits and sometimes fail. A production client uses timeouts, retries with exponential backoff and jitter on retryable errors such as 429 and 5xx, and never retries errors a retry cannot fix, such as 401 or 400."

SPEAK THE LANGUAGE

Vocabulary and today's map

Provider	The service that runs the model (a hosted API or a local server like Ollama).
OpenAI-compatible API	An HTTP API with the same request and response shape as OpenAI's /chat/completions. One client covers many providers.
Token	The unit models read and write; roughly 4 characters of English on average. Billing and limits are in tokens.
Streaming	Sending the answer in pieces as it is generated instead of waiting for the full answer.
SSE	Server-sent events: a simple HTTP format for streaming from server to client. Lines start with `data:` and events end with a blank line.
Timeout	The maximum time you wait for a step before giving up.
Retry	Trying a failed call again.
Exponential backoff	Waiting longer after each failure: up to 0.5 s, 1 s, 2 s, 4 s…
Jitter	Randomising the wait so many clients do not retry at the same moment.
Idempotent	Safe to repeat: doing it twice has the same effect as once.
Fallback model	A second model that answers when the main one is unavailable.



The route depends only on the interface, so switching provider, adding a fallback or running tests offline needs no change to the route.

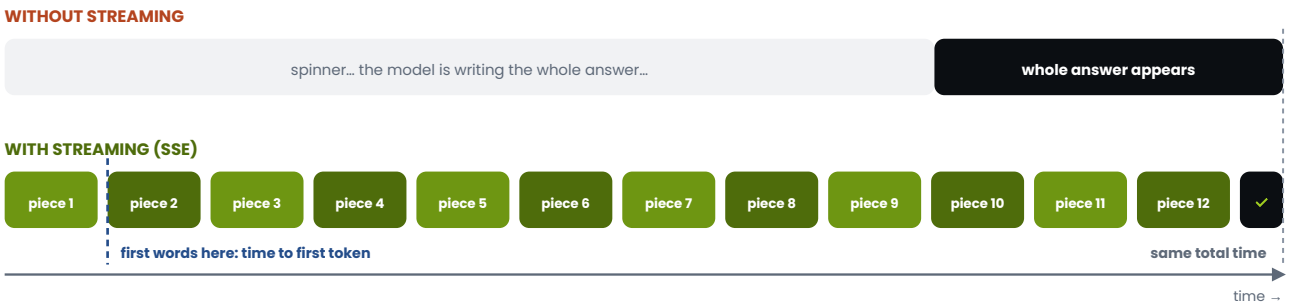
INTERVIEW MODE

"What if your provider goes down, raises prices or rate-limits you?" → "I swap or fall back by configuration; the business logic does not change."

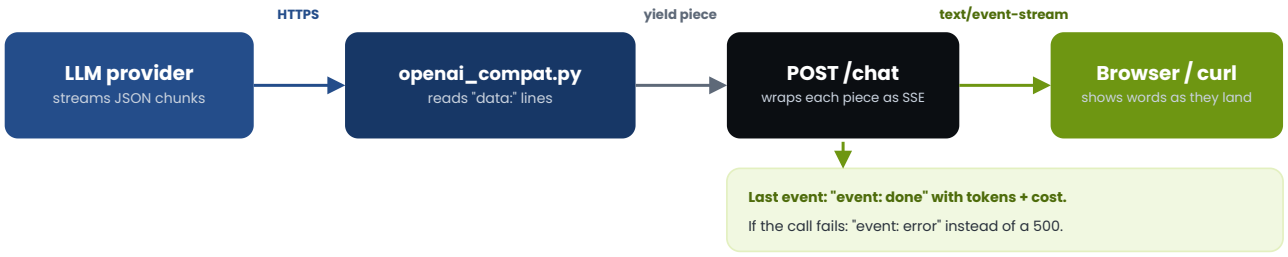
SAME TOTAL TIME, FASTER FIRST WORD

Streaming with SSE

Without streaming, the user stares at a spinner until the whole answer is generated. With streaming, the first words appear quickly. Total time is the same; **perceived** latency drops.



Illustration, not a measurement: the total time is the same; only when the first words appear changes.



THE WIRE FORMAT WE SEND

```
data: {"delta": "Hel"}

data: {"delta": "lo"}

event: done
data: {"usage": {"input_tokens": 7,
"output_tokens": 2,
       "estimated": false}, "cost_usd":
0.0000065}
```

Read it like this

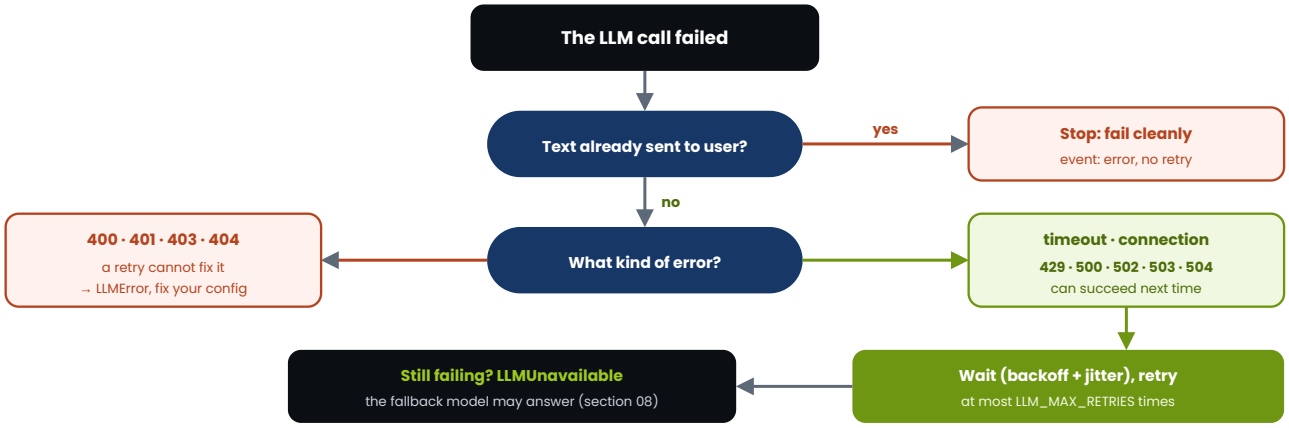
Each event is `data: <json>` followed by a **blank line**. A named event (`event: done`) tells the client the stream is complete and carries the token usage and cost.

MEMORY HOOK

"data, blank, data, blank, done." Forget the blank line and the browser never sees the event end.

WHICH FAILURES DESERVE A SECOND TRY?

Retry only what can succeed



The five rules we code today

- [ ] **Timeout every call.** A call with no timeout can hang forever and tie up your server.
- [ ] **Retry only** timeouts, connection errors, 429 and 500/502/503/504.
- [ ] **Never retry** 400, 401/403, 404: a retry cannot fix them and wastes quota.
- [ ] **Never retry after text was streamed:** the user would see the answer start again.
- [ ] **Back off with full jitter:** wait random(0, min(cap, base × $2^{attempt}$)).

PAUSE AND PREDICT

Your key is wrong and the provider returns 401. With 3 retries, how many calls does the doc's client make?

Answer: exactly 1. 401 raises LLMError straight away. A test proves it: `test_does_not_retry_on_401` .

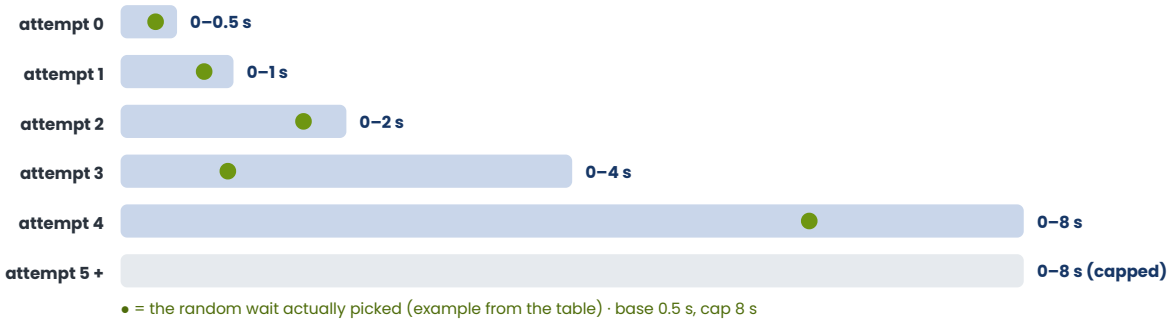
WHAT GOES WRONG

A `try/except` that retries *everything* 5 times turns one bad key into 6 wasted calls, and turns one provider outage into a retry storm from every user at once.

WAIT LONGER, WAIT RANDOMLY

Exponential backoff with full jitter

Worked numbers with **base 0.5 s** and **cap 8 s**. The window doubles each attempt; the actual wait is a random point inside it.



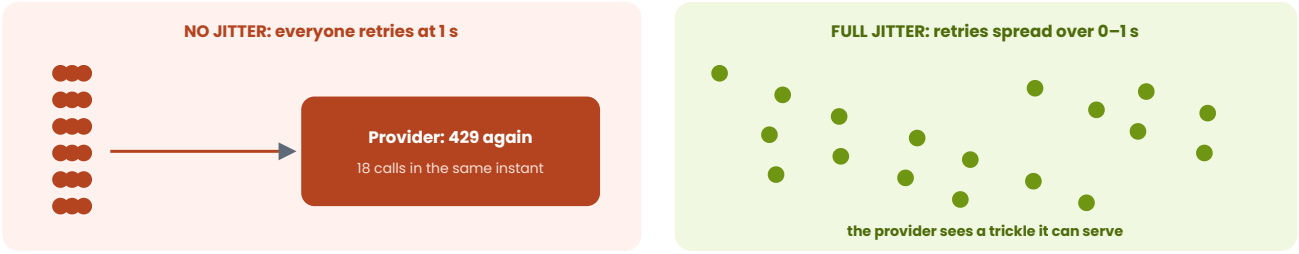
ATTEMPT	WINDOW	EXAMPLE WAIT
0	0–0.5 s	0.31 s
1	0–1 s	0.74 s
2	0–2 s	1.62 s
3	0–4 s	0.95 s
4	0–8 s	6.10 s
5+	0–8 s (capped)	…

Worst case for a user

With 3 retries the most you can wait in backoff is **0.5 + 1 + 2 = 3.5 s**, plus the time of each attempt. That is why a user-facing request keeps retries small.

MEMORY HOOK

"Double the window, pick a random seat."



JUST FOR FUN

Jitter is just politely not walking through the door at the exact same moment as 89 other people. Engineering, but make it manners.

ONE CLIENT FOR MANY PROVIDERS

The streaming client · part 1

Create `app/llm/openai_compat.py`. Type it; do not paste blindly. Part 1: which errors are retryable, and the backoff function from section 06.

APP/LLM/OPENAI_COMPAT.PY · LINES 1–24

```
"""Streaming client for any OpenAI-compatible /chat/completions endpoint.

Reliability: per-call timeout, retries with exponential backoff and full jitter on
timeouts, connection errors, HTTP 429 and HTTP 5xx. Client errors (400, 401, 403, 404)
are not retried because retrying cannot fix them.
"""
import asyncio
import json
import random
from collections.abc import AsyncIterator

import httpx

from app.llm.base import LLMError, LLMUnavailable, StreamResult
from app.schemas import Message, Usage
from app.tokens import estimate_tokens

RETRYABLE_STATUS = {429, 500, 502, 503, 504}


def backoff_delay(attempt: int, base: float, cap: float = 8.0) -> float:
    """Full jitter: a random wait between 0 and base * 2**attempt (capped)."""
    return random.uniform(0, min(cap, base * (2**attempt)))
```

PAUSE AND PREDICT

`backoff_delay(3, base=0.5)` : what range can it return?

Answer: 0 to 4 seconds. $0.5 \times 2^3 = 4$, below the 8 s cap, and `random.uniform` picks any point inside.

MEMORY HOOK

RETRYABLE_STATUS is a whitelist. Anything not on it is not retried: safer than listing what to skip.

SETTINGS COME IN, TIMEOUTS ARE SET

The streaming client · part 2

The constructor takes everything from settings. `transport` lets tests swap the real network for a fake one.

APP/LLM/OPENAI_COMPAT.PY · LINES 25–52

```
class OpenAICompatProvider:
    name = "openai_compat"

    def __init__(
        self,
        base_url: str,
        api_key: str,
        model: str,
        timeout_seconds: float = 30.0,
        max_retries: int = 3,
        backoff_base_seconds: float = 0.5,
        transport: httpx.AsyncBaseTransport | None = None,
    ) -> None:
        self.base_url = base_url.rstrip("/")
        self.api_key = api_key
        self.model = model
        self.timeout = httpx.Timeout(timeout_seconds, connect=5.0)
        self.max_retries = max_retries
        self.backoff_base = backoff_base_seconds
        self.transport = transport  # injected in tests

    def _headers(self) -> dict[str, str]:
        headers = {"Content-Type": "application/json"}
        if self.api_key:
            headers["Authorization"] = f"Bearer {self.api_key}"
        return headers
```

PAUSE AND PREDICT

`httpx.Timeout(30, connect=5)` : does 30 mean the whole answer must finish in 30 seconds?

Answer: no. It is the longest httpx will wait for the next bytes (and 5 s to connect). A stream that keeps sending words can run longer than 30 s; a silent one fails after 30 s.

SEND THE REQUEST, CHECK THE STATUS

The streaming client · part 3

The request asks for `"stream": true` and a final usage chunk. Retryable statuses become an exception the retry loop catches; any other 4xx becomes `LLMError` at once.

APP/LLM/OPENAI_COMPAT.PY · LINES 53–86

```
    async def stream_chat(
        self, messages: list[Message], temperature: float, result: StreamResult
    ) -> AsyncIterator[str]:
        payload = {
            "model": self.model,
            "messages": [m.model_dump() for m in messages],
            "temperature": temperature,
            "stream": True,
            # Ask for a final usage chunk; providers that do not support it ignore or reject it.
            "stream_options": {"include_usage": True},
        }
        last_error: Exception | None = None
        for attempt in range(self.max_retries + 1):
            produced_any = False
            try:
                async with httpx.AsyncClient(timeout=self.timeout, transport=self.transport) as client:
                    async with client.stream(
                        "POST", f"{self.base_url}/chat/completions",
                        headers=self._headers(), json=payload,
                    ) as response:
                        if response.status_code in RETRYABLE_STATUS:
                            raise httpx.HTTPStatusError(
                                f"retryable status {response.status_code}",
                                request=response.request, response=response,
                            )
                        if response.status_code >= 400:
                            body = (await response.aread()).decode(errors="replace")[:300]
                            raise LLMError(f"provider returned {response.status_code}: {body}")
                        text_parts: list[str] = []
                        async for line in response.aiter_lines():
                            if not line.startswith("data:"):
                                continue
                            data = line[len("data:"):].strip()
                            if data == "[DONE]":
```

WHAT GOES WRONG

Some providers reject `stream_options` with a 400. Remove that field for that provider; usage will then be estimated.

READ THE STREAM, RETRY SAFELY

The streaming client · part 4

APP/LLM/OPENAI_COMPAT.PY · LINES 87–119

```
                            break
                        chunk = json.loads(data)
                        if chunk.get("usage"):
                            u = chunk["usage"]
                            result.usage = Usage(
                                input_tokens=u.get("prompt_tokens", 0),
                                output_tokens=u.get("completion_tokens", 0),
                            )
                        for choice in chunk.get("choices", []):
                            piece = (choice.get("delta") or {}).get("content")
                            if piece:
                                produced_any = True
                                text_parts.append(piece)
                                yield piece
                    if result.usage.input_tokens == 0 and result.usage.output_tokens == 0:
                        result.usage = Usage(
                            input_tokens=sum(estimate_tokens(m.content) for m in messages),
                            output_tokens=estimate_tokens("".join(text_parts)),
                            estimated=True,
                        )
                    return
            except LLMError:
                raise
            except (httpx.TimeoutException, httpx.TransportError, httpx.HTTPStatusError) as exc:
                # Never retry after text has already been streamed to the user:
                # a retry would repeat or duplicate the answer.
                if produced_any:
                    raise LLMError(f"stream broke mid-answer: {exc}") from exc
                last_error = exc
                if attempt < self.max_retries:
                    await asyncio.sleep(backoff_delay(attempt, self.backoff_base))
        # Session 5: LLMUnavailable (a kind of LLMError) so the fallback knows it may step in.
        raise LLMUnavailable(f"LLM call failed after {self.max_retries + 1} attempts: {last_error}")
```

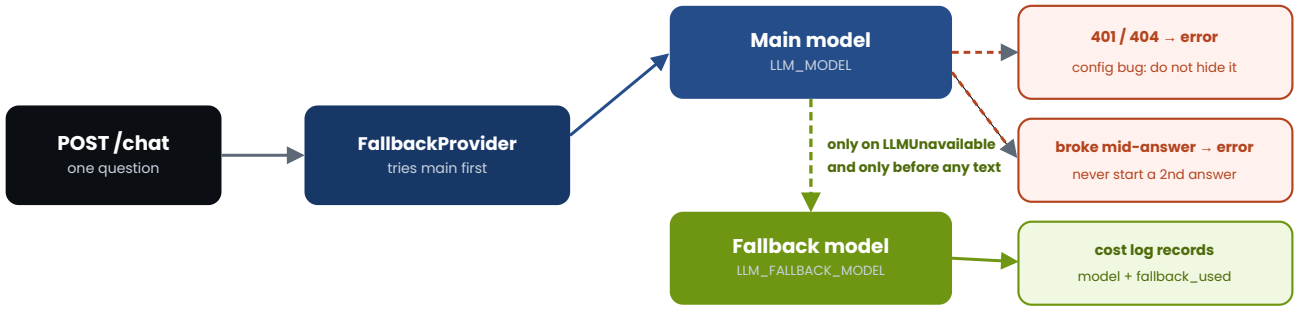
Find each rule in the code

Timeout	`httpx.Timeout(timeout_seconds, connect=5.0)`
Retryable statuses	`RETRYABLE_STATUS`
No retry on 4xx	`raise LLMError(...)` before the except that retries
No retry after streaming	the `produced_any` check
Backoff with jitter	`backoff_delay`
Fallback signal	`raise LLMUnavailable(...)` after the last attempt ADDED FOR THIS SESSION

WHEN THE MAIN MODEL IS DOWN

A second model steps in ADDED FOR THIS SESSION

Retries handle a blip. If the main model is still unavailable after every retry, a **fallback model** answers the same question, often a different model or a different provider.



Two times we do NOT fall back

401, 403, 404, 400: your key, model name or request is wrong. Falling back would hide a bug you must fix.

After text reached the user: a second model would start a new, different answer in the middle of the first one. Fail cleanly instead.

PAUSE AND PREDICT

Timeout 30 s, 3 retries, and the main provider goes silent on every attempt. How long can the user wait before the fallback even starts?

Answer: up to about 2 minutes: 4 attempts × 30 s + 3.5 s of backoff = 123.5 s. Our suggestion when you set a fallback: `LLM_MAX_RETRIES=1` , so 2 × 30 s + up to 0.5 s ≈ 1 minute; lower the timeout too if your answers are short.

INTERVIEW MODE

"I retry transient errors with capped backoff. If the main model is still unavailable and nothing has been streamed yet, a fallback model answers, and the cost log records which model answered. Config errors like 401 are never hidden by the fallback."

A NEW ERROR TYPE

Fallback in code · part 1 ADDED FOR THIS SESSION

Two additions to `app/llm/base.py`. One change from the GenAI 02 reference code: the streaming client's last line raises **LLMUnavailable** (a kind of LLMError) instead of a plain LLMError, so the fallback can tell "provider down" apart from "your key is wrong".

APP/LLM/BASE.PY · ADD TO STREAMRESULT, AND A NEW ERROR CLASS AT THE END

```
class StreamResult:
    """Collects usage while a stream is consumed (filled in by the provider)."""

    def __init__(self) -> None:
        self.usage = Usage()
        self.model = ""  # added for Session 5: which model actually answered
        self.fallback_used = False  # added for Session 5

# ... LLMProvider and LLMError stay as they are ...

class LLMUnavailable(LLMError):
    """Added for Session 5: the provider was down, slow or rate-limited after every retry.
    Only this error switches to the fallback model; a bad key (401) or wrong model (404)
    is a configuration bug that the fallback must not hide."""
```

PAUSE AND PREDICT

LLMUnavailable is a subclass of LLMError. Does the /chat route need a change to catch it?

Answer: no. `except LLMError` also catches every subclass, so the route still turns it into `event: error` when there is no fallback.

MEMORY HOOK

"Unavailable = try someone else. Error = fix yourself."

A WRAPPER THAT IS ALSO A PROVIDER

Fallback in code · part 2 ADDED FOR THIS SESSION

The wrapper has the same `stream_chat` method as every provider, so the route does not change at all.

APP/LLM/FALLBACK.PY · NEW FILE

```
"""Added for Session 5: answer with a second model when the main one is unavailable."""
from collections.abc import AsyncIterator

from app.llm.base import LLMUnavailable, StreamResult
from app.schemas import Message


class FallbackProvider:
    def __init__(self, main, fallback) -> None:
        self.main = main
        self.fallback = fallback
        self.name = main.name
        self.model = main.model

    async def stream_chat(
        self, messages: list[Message], temperature: float, result: StreamResult
    ) -> AsyncIterator[str]:
        produced_any = False
        try:
            async for piece in self.main.stream_chat(messages, temperature, result):
                produced_any = True
                yield piece
            result.model = self.main.model
            return
        except LLMUnavailable:
            # No fallback once text reached the user: a second model would start a new,
            # different answer in the middle of the first one.
            if produced_any:
                raise
        result.fallback_used = True
        result.model = self.fallback.model
        async for piece in self.fallback.stream_chat(messages, temperature, result):
            yield piece
```

SWITCH IT ON FROM .ENV

Fallback in code · part 3 ADDED FOR THIS SESSION

The factory wraps the main provider only when `LLM_FALLBACK_MODEL` is set. Leave it empty and nothing changes.

APP/LLM/FACTORY.PY · THE OPENAI_COMPAT BRANCH

```
    if settings.llm_provider == "openai_compat":
        main = OpenAICompatProvider(
            base_url=settings.llm_base_url,
            api_key=settings.llm_api_key,
            model=settings.llm_model,
            timeout_seconds=settings.llm_timeout_seconds,
            max_retries=settings.llm_max_retries,
            backoff_base_seconds=settings.llm_backoff_base_seconds,
        )
        if not settings.llm_fallback_model:  # added for Session 5
            return main
        backup = OpenAICompatProvider(
            base_url=settings.llm_fallback_base_url or settings.llm_base_url,
            api_key=settings.llm_fallback_api_key or settings.llm_api_key,
            model=settings.llm_fallback_model,
            timeout_seconds=settings.llm_timeout_seconds,
            max_retries=1,  # the user has already waited for the main model
            backoff_base_seconds=settings.llm_backoff_base_seconds,
        )
        return FallbackProvider(main, backup)
    raise ValueError(f"unknown LLM_PROVIDER: {settings.llm_provider}")
```

APP/CONFIG.PY · ADD TO SETTINGS

```
    # Added for Session 5: a second model that answers when the main one is unavailable.
    # Empty LLM_FALLBACK_MODEL = no fallback. Base URL and key default to the main ones.
    llm_fallback_model: str = ""
    llm_fallback_base_url: str = ""
    llm_fallback_api_key: str = ""
```

.ENV.EXAMPLE

```
# .env.example: add under the LLM settings (added for this session)
# Second model used only when the main one is unavailable. Empty = no fallback.
# Base URL and key default to the main ones (same provider, another model).
LLM_FALLBACK_MODEL=
LLM_FALLBACK_BASE_URL=
LLM_FALLBACK_API_KEY=
```

PAUSE AND PREDICT

Why does the backup get `max_retries=1` instead of 3?

Answer: by the time the backup runs, the user has already waited for every retry of the main model. One retry keeps the total wait bounded.

WHAT DOES ONE QUESTION COST?

Tokens × price = cost

Cost of one call = input tokens × input price + output tokens × output price. Prices are quoted per million tokens. Worked example with hypothetical prices:



PAUSE AND PREDICT

Which number would you cut first to reduce cost: the 1,200 input tokens or the 300 output tokens?

Usually the input. Retrieved chunks are the biggest part of the prompt, so sending 4 good chunks instead of 10 mediocre ones cuts cost and often improves accuracy. Output tokens are often priced higher per token, so long answers matter too.

ESTIMATES ARE NOT BILLS

When the provider does not report usage, we estimate about 4 characters per token and mark the record `usage_estimated: true`. Estimates are for logging only, never for billing.

Same engine, your product

REG	Long circulars → big retrieved chunks → input tokens dominate the bill.
ENT	Many clients share one app: later you can total /usage per client too.
FIN	Annual-report tables are long; sending 4 good chunks instead of 10 is your biggest saving.
CODE	Code is dense with symbols, so it rarely matches the 4-characters rule. Trust the provider's usage numbers.
CIT	Hindi, Tamil and Telugu text usually needs more tokens than English for the same meaning, so the 4-character estimate is low. Trust the provider's numbers.

ONE JSON LINE PER CALL

Cost logging

APP/COST.PY

```python
"""Cost logging: one JSON line per LLM call."""
import json
import time
from pathlib import Path

from app.schemas import Usage


def cost_usd(usage: Usage, price_in_per_m: float, price_out_per_m: float) -> float:
    return (usage.input_tokens * price_in_per_m + usage.output_tokens * price_out_per_m) / 1_000_000


def log_call(path: str, *, provider: str, model: str, usage: Usage, latency_ms: float,
             price_in_per_m: float, price_out_per_m: float, ok: bool,
             fallback_used: bool = False) -> dict:
    record = {
        "ts": round(time.time(), 3),
        "provider": provider,
        "model": model,
        "input_tokens": usage.input_tokens,
        "output_tokens": usage.output_tokens,
        "usage_estimated": usage.estimated,
        "latency_ms": round(latency_ms, 1),
        "cost_usd": round(cost_usd(usage, price_in_per_m, price_out_per_m), 8),
        "ok": ok,
        "fallback_used": fallback_used,  # added for Session 5
    }
    p = Path(path)
    p.parent.mkdir(parents=True, exist_ok=True)
    with p.open("a", encoding="utf-8") as f:
        f.write(json.dumps(record) + "\n")
    return record
```

One addition to the GenAI 02 reference code: the `fallback_used` argument and field **ADDED FOR THIS SESSION**. Older calls without it still work, because it defaults to False.

PAUSE AND PREDICT

Why does `log_call` open the file in `"a"` mode and write one line per call?

Answer: append-only JSON lines are cheap to write, never rewrite old data, and are easy to read back line by line, which is exactly what GET /usage does.

SETTINGS AND THE PROVIDER, INJECTED

The app setup

Replace `app/main.py`. Part 1: imports, and FastAPI dependencies that build the settings and the provider for every request. Tests swap them with `dependency_overrides`.

APP/MAIN.PY · LINES 1–30

```python
import json
import time
from typing import Annotated

from fastapi import Depends, FastAPI
from fastapi.responses import StreamingResponse

from app.config import Settings, get_settings
from app.cost import log_call
from app.llm.base import LLMError, StreamResult
from app.llm.factory import build_provider
from app.schemas import ChatRequest
from app.usage import usage_by_day

app = FastAPI(title="Examate RAG starter", version="0.1.0")

SettingsDep = Annotated[Settings, Depends(get_settings)]


def get_provider(settings: SettingsDep):
    return build_provider(settings)


ProviderDep = Annotated[object, Depends(get_provider)]


@app.get("/health")
def health(settings: SettingsDep) -> dict:
    """Liveness: the process is up. Must be fast and must not call the LLM or the DB."""
    return {"status": "ok", "app": settings.app_name, "environment": settings.environment}
```

INTERVIEW MODE

"The route receives the provider through dependency injection, so tests can replace it with a fake and production can switch it by configuration."

STREAM, LOG, AND TELL THE CLIENT

The /chat route

The route streams each piece as an SSE event, turns a failure into `event: error` instead of a 500, logs the call, then sends `event: done` with tokens and cost.

APP/MAIN.PY · LINES 31–62

```python
def sse(data: dict, event: str | None = None) -> str:
    prefix = f"event: {event}\n" if event else ""
    return f"{prefix}data: {json.dumps(data)}\n\n"


@app.post("/chat")
async def chat(req: ChatRequest, settings: SettingsDep, provider: ProviderDep):
    """Streams the answer as Server-Sent Events (SSE) and logs tokens and cost per call."""

    async def event_stream():
        result = StreamResult()
        start = time.perf_counter()
        ok = True
        try:
            async for piece in provider.stream_chat(req.messages, req.temperature, result):
                yield sse({"delta": piece})
        except LLMError as exc:
            ok = False
            yield sse({"error": str(exc)}, event="error")
        latency_ms = (time.perf_counter() - start) * 1000
        record = log_call(
            settings.cost_log_path, provider=provider.name, model=result.model or provider.model,
            usage=result.usage, latency_ms=latency_ms,
            price_in_per_m=settings.price_input_per_million,
            price_out_per_m=settings.price_output_per_million, ok=ok,
            fallback_used=result.fallback_used,
        )
        yield sse({"usage": result.usage.model_dump(), "cost_usd": record["cost_usd"]}, event="done")

    return StreamingResponse(event_stream(), media_type="text/event-stream")
```

The GenAI 02 reference also has a `/ready` route that checks the database. You add it in GenAI 6 together with Docker Compose and Postgres. `model=result.model or provider.model` and `fallback_used=…` are **ADDED FOR THIS SESSION**.

PAUSE AND PREDICT

The provider fails after 3 words. What does the browser receive?

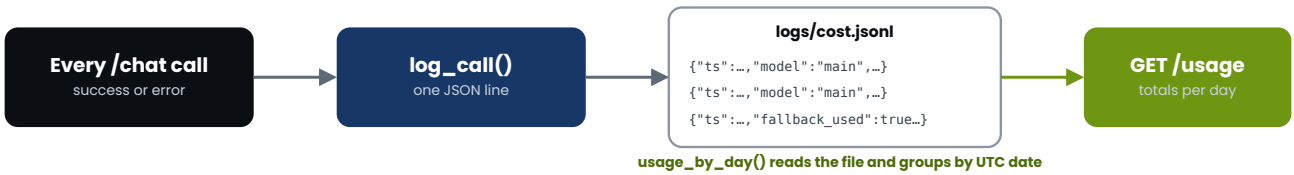
Answer: three `data:` events, then `event: error`, then `event: done` with the usage so far. The HTTP status is still 200, because the stream had already started; the error lives inside the stream.

FROM LOG LINES TO A DAILY BILL

Log every call, add GET /usage

ADDED FOR THIS SESSION

Every call, success or error, writes one line to `logs/cost.jsonl`. `GET /usage` reads that file and returns calls, tokens and cost per day (UTC date).



APP/USAGE.PY · NEW FILE

```
"""Added for Session 5: totals per day (UTC) from logs/cost.jsonl, for GET /usage."""
import json
from datetime import UTC, datetime
from pathlib import Path


def usage_by_day(path: str) -> dict[str, dict]:
    days: dict[str, dict] = {}
    p = Path(path)
    if not p.exists():
        return days
    for line in p.read_text(encoding="utf-8").splitlines():
        try:
            r = json.loads(line)
        except json.JSONDecodeError:
            continue  # skip a blank or half-written line instead of failing the whole report
        day = datetime.fromtimestamp(r["ts"], tz=UTC).date().isoformat()
        d = days.setdefault(day, {"calls": 0, "input_tokens": 0, "output_tokens": 0, "cost_usd": 0.0})
        d["calls"] += 1
        d["input_tokens"] += r["input_tokens"]
        d["output_tokens"] += r["output_tokens"]
        d["cost_usd"] = round(d["cost_usd"] + r["cost_usd"], 8)
    return days
```

APP/MAIN.PY · ADD AT THE END

```
@app.get("/usage")
def usage(settings: SettingsDep) -> dict:
    """Added for Session 5: calls, tokens and cost
per day (UTC) from the cost log."""
    return usage_by_day(settings.cost_log_path)
```

EXAMPLE RESPONSE

```
{"2026-10-24": {"calls": 2, "input_tokens": 2000,
                "output_tokens": 500, "cost_usd":
0.00175},
 "2026-10-25": {"calls": 1, "input_tokens": 100,
                "output_tokens": 50, "cost_usd":
0.000125}}
```

BEFORE A REAL LAUNCH

Anyone with your URL can open `/usage`. It is fine for this project; in a real product you would protect it with an admin key or keep it internal.

PROVE IT WITHOUT A KEY

Tests: 14 passed

Keep the four test files from the GenAI 02 reference (copy them from the trainer's reference folder or type them from that doc). They fake the provider with `httpx.MockTransport`: a 503 that succeeds on the third try, a 401 that must not be retried, a timeout that gives up. Then add one new file for today.

TESTS/TEST_FALLBACK_AND_USAGE.PY · LINES 1–38 (NEW FILE, ADDED FOR THIS SESSION)

```
"""Added for Session 5: fallback model and GET /usage."""
import asyncio
import json

import httpx
import pytest

from app.llm.base import LLMError, StreamResult
from app.llm.fake import FakeProvider
from app.llm.fallback import FallbackProvider
from app.llm.openai_compat import OpenAICompatProvider
from app.schemas import Message

MSGS = [Message(role="user", content="hello")]


def slow_main(handler):
    return OpenAICompatProvider(base_url="http://test/v1", api_key="k", model="main-model",
                                max_retries=1, backoff_base_seconds=0.0,
                                transport=httpx.MockTransport(handler))


def collect(p):
    result = StreamResult()

    async def run():
        return "".join([piece async for piece in p.stream_chat(MSGS, 0.2, result)])

    return asyncio.run(run()), result


def test_fallback_answers_when_main_times_out():
    def handler(request):
        raise httpx.ReadTimeout("slow", request=request)

    text, result = collect(FallbackProvider(slow_main(handler), FakeProvider(model="backup-model")))
    assert text.startswith("You asked: hello")
    assert result.fallback_used is True and result.model == "backup-model"
```

PAUSE AND PREDICT

In `slow_main`, why `backoff_base_seconds=0.0`?

Answer: the test should prove the logic, not sleep. With base 0 every wait is 0 s, so the whole suite runs in well under a second.

NO FALLBACK ON A BAD KEY; TOTALS PER DAY

Tests · part 2

TESTS/TEST_FALLBACK_AND_USAGE.PY · LINES 39–68

```python
def test_no_fallback_on_bad_key():
    def handler(request):
        return httpx.Response(401, json={"error": "bad key"})

    with pytest.raises(LLMError):
        collect(FallbackProvider(slow_main(handler), FakeProvider(model="backup-model")))


def test_main_answers_without_fallback():
    text, result = collect(FallbackProvider(FakeProvider(model="main-model"), FakeProvider(model="backup")))
    assert result.fallback_used is False and result.model == "main-model"


def test_usage_totals_per_day(client, tmp_path):
    log = [
        {"ts": 1792800000.0, "input_tokens": 1200, "output_tokens": 300, "cost_usd": 0.00105},  # 2026-10-24
        {"ts": 1792810000.0, "input_tokens": 800, "output_tokens": 200, "cost_usd": 0.0007},    # 2026-10-24
        {"ts": 1792900000.0, "input_tokens": 100, "output_tokens": 50, "cost_usd": 0.000125},   # 2026-10-25
    ]
    path = client.settings.cost_log_path
    with open(path, "w", encoding="utf-8") as f:
        f.write("\n".join(json.dumps(r) for r in log) + "\n{half-written line\n")
    r = client.get("/usage")
    assert r.status_code == 200
    assert r.json() == {
        "2026-10-24": {"calls": 2, "input_tokens": 2000, "output_tokens": 500, "cost_usd": 0.00175},
        "2026-10-25": {"calls": 1, "input_tokens": 100, "output_tokens": 50, "cost_usd": 0.000125},
    }
```

PAUSE AND PREDICT

The /usage test writes a broken line `{half-written line` into the log. Why?

Answer: if the server crashes mid-write, the last line can be half written. `usage_by_day` skips it instead of failing the whole report, and the test proves that.

ONE FIXTURE, THEN RUN EVERYTHING

Tests · part 3

TESTS/CONFTEST.PY · REPLACE WITH

```
import pytest
from fastapi.testclient import TestClient

from app.config import Settings, get_settings
from app.main import app


@pytest.fixture
def client(tmp_path):
    settings = Settings(app_name="examate-rag", environment="test", llm_provider="fake",
                        cost_log_path=str(tmp_path / "cost.jsonl"),
                        price_input_per_million=0.5, price_output_per_million=1.5)
    app.dependency_overrides[get_settings] = lambda: settings
    with TestClient(app) as c:
        c.settings = settings
        yield c
    app.dependency_overrides.clear()
```

RUN

```
pip install httpx==0.28.1     # also add it to requirements.txt
ruff check .
pytest -q
# All checks passed!
# 14 passed
```

Where the 14 come from

/health 1 · test_chat 2 · test_openai_compat 5 · test_cost_and_schemas 2 · test_fallback_and_usage 4. The four older test files come from the GenAI 02 reference; this conftest also sets `environment="test"` so the template's /health test still passes.

WHAT GOES WRONG

Tests suddenly call a real provider and fail on the train Wi-Fi? Your `.env` leaked into the tests. The fixture above forces `llm_provider="fake"`; keep it.

SEE IT STREAM

Run it with your questions

First with the fake provider (no key), then with the real model your trainer names. Use a question from your own `docs/QUESTIONS.md`.

WITH THE FAKE PROVIDER

```
uvicorn app.main:app --reload

curl -N -X POST localhost:8000/chat -H "Content-Type: application/json" \
  -d '{"messages":[{"role":"user","content":"What is pgvector?"}]}'

curl localhost:8000/usage
```

You should see `data: {"delta": …}` lines arrive one by one, then `event: done` with usage, and a new line in logs/cost.jsonl. `-N` tells curl not to buffer.

.ENV FOR A REAL MODEL (EXAMPLES; CHECK EACH PROVIDER'S PAGE FOR CURRENT URLS AND MODELS)

```
LLM_PROVIDER=openai_compat
LLM_API_KEY=your-key-here            # never commit .env
LLM_MODEL=model-id-from-the-provider
# Gemini:     LLM_BASE_URL=https://generativelanguage.googleapis.com/v1beta/openai/
# Groq:       LLM_BASE_URL=https://api.groq.com/openai/v1
# OpenRouter: LLM_BASE_URL=https://openrouter.ai/api/v1
# Ollama:     LLM_BASE_URL=http://localhost:11434/v1
PRICE_INPUT_PER_MILLION=0      # from the pricing page; 0 = free tier/local
PRICE_OUTPUT_PER_MILLION=0
LLM_FALLBACK_MODEL=            # optional: a second model id
```

COMMIT

```
git add -A
git commit -m "feat: streaming /chat, safe retries, fallback model, cost log, GET /usage"
git push
```



INTERVIEW MODE

"I implemented the provider interface, a fake provider for tests, an OpenAI-compatible streaming client with timeouts and safe retries, a fallback model, per-call cost logging, an SSE endpoint and a /usage report, all covered by tests that simulate rate limits, bad keys and timeouts."

FIX IT YOURSELF FIRST

Common errors and fixes

SYMPTOM	CAUSE	FIX
401 from provider	Wrong or missing key, or a key for a different provider	Check LLM_API_KEY and LLM_BASE_URL belong together. Not retried, and no fallback, by design.
404 from provider	Wrong model name or base URL path	Copy the model ID from the provider's model list; check the URL ends at /v1 (or the provider's documented base).
400 mentioning stream_options	Provider does not accept stream_options	Remove that field for that provider; usage will then be estimated.
Output arrives all at once	A proxy or your client buffers the response	Test with `curl -N`; in the frontend read the stream piece by piece.
ConnectError to localhost:11434	Ollama not running	Start Ollama and pull the model, or use a hosted provider.
Tests call the real provider	LLM_PROVIDER leaked from your .env	Tests override settings in conftest.py; keep that fixture.
Fallback never used	LLM_FALLBACK_MODEL empty, or the error was a 401/404	Set the variable; remember config errors do not fall back.
/usage returns {}	No calls yet, or a different COST_LOG_PATH	Make one /chat call; check the path in .env.
ModuleNotFoundError: httpx	httpx only in dev requirements	Add `httpx==0.28.1` to requirements.txt; it is now a runtime dependency.

Checkpoints before you leave

- [ ] `pytest -q` shows 14 passed with the fake provider (no key needed)
- [ ] A real model (or the fake one, if you have no key yet) streams one of YOUR questions through /chat
- [ ] `logs/cost.jsonl` has one line per call with tokens, latency, cost and fallback_used
- [ ] `GET /usage` shows today's calls
- [ ] You can point to the exact line that prevents retrying a 401

PRACTISE ALOUD WITH A PARTNER

Interview questions on today

PATTERN-ONLY = appears in public GenAI interview question compilations without a named company and date, so no company is claimed.

Q1. How do you handle rate limits and failures from an LLM API?

PATTERN-ONLY

> **Model answer.** "Every call has a timeout. I retry only retryable failures (timeouts, connection errors, 429 and 5xx) with exponential backoff and full jitter, a small retry cap, and never after I've started streaming to the user. I don't retry 400, 401, 403 or 404. Behind the retries I add a fallback model, per-user rate limits so one user can't exhaust the quota, and caching for repeated questions."

Follow-up 1. Why jitter? → Without it, many clients that failed together retry together and fail again in synchronised waves.

Follow-up 2. Base 0.5 s, cap 8 s, 3 retries: worst-case waiting? → 0.5 + 1 + 2 = 3.5 s plus attempt durations; fine for chat, too long for autocomplete.

Follow-up 3. 90 users share one key with a 30 requests-per-minute limit? → Queue requests server-side with a token-bucket limiter at 30/min, cache, route easy questions to a model with higher limits, or get more keys per tenant where allowed.

Weak: "Wrap it in try/except and retry."

Strong: "Timeouts, retry only 429/5xx with capped backoff and jitter, no retry after streaming, fallback and rate limiting."

Q2. Why stream LLM responses, and how?

PATTERN-ONLY

> **Model answer.** "Generation is token by token and often takes seconds. Streaming sends tokens as they're produced, so time to first token drops from the full generation time to a fraction of a second, even though total time is the same. I use server-sent events: one-way, plain HTTP, easy to proxy. WebSockets are for two-way interaction."

Follow-up 1. What breaks streaming in production? → Proxies or load balancers that buffer responses, and timeouts on long-lived connections.

Follow-up 2. How do you send token usage when streaming? → A final event with usage, from the provider's usage chunk or an estimate.

Follow-up 3. Client disconnects mid-answer? → Stop generating to save tokens; log a partial completion.

Weak: "Streaming is faster."

Strong: "Same total time, much lower time to first token; SSE for one-way; watch buffering proxies."

THREE MORE, THEN THE 60-SECOND ANSWER

Interview questions 3 to 5

Q3. Which HTTP errors should a client retry?

PATTERN-ONLY

> **Model answer.** "Retry transient ones: timeouts, connection resets, 429 and 502/503/504, and usually 500. Respect a Retry-After header when the server sends one. Don't retry 400, 401, 403, 404 or validation errors. Only retry operations that are safe to repeat."

Follow-up 1. Is an LLM call idempotent? → Retrying is safe for correctness before any output reached the user, but it costs tokens and may give a different answer.

Follow-up 2. Why cap retries? → Unbounded retries multiply load during an outage (retry storms).

Follow-up 3. What is a circuit breaker? → After repeated failures, stop calling the provider for a cool-down period and fail fast or use the fallback (System Design Lesson 11).

Weak: "Retry everything 5 times."

Strong: "Retry transient errors with backoff, respect Retry-After, never 4xx client errors, cap retries."

Q4. How much does one question cost in your app?

PATTERN-ONLY

> **Model answer.** "I log input tokens, output tokens and latency for every call and multiply by the per-million prices. For example, 1,200 input and 300 output tokens at $0.50 and $1.50 per million is about $0.001 per question. Most of the input is retrieved context, so retrieving fewer, better chunks is my first lever, then caching, then a cheaper model for easy questions."

Follow-up 1. Which is usually more expensive per token, input or output? → Output, for most hosted models.

Follow-up 2. At 1 million questions a month? → About $1,050 at those prices; caching 30% of questions saves about $315.

Follow-up 3. Free tier in production? → No: limits and terms change, and there is no service guarantee; plan a paid or self-hosted path.

Weak: "It's free, I use a free tier."

Strong: "Measured tokens per call times price; levers are context size, caching and model routing."

Q5 (revision from Monday). How do you get reliable JSON out of an LLM?

PATTERN-ONLY

> **Model answer.** "I give a schema, use the provider's JSON or structured-output mode when available, and always validate with Pydantic. On validation failure I retry once with the error message included, then fall back. I never let unvalidated output trigger actions."

Follow-up 1. Why validate if the provider has a JSON mode? → JSON mode guarantees syntax, not that the values make sense (for example confidence between 0 and 1).

Weak: "Ask the model nicely to return JSON."

Strong: "Schema + structured mode + Pydantic validation + retry with error + fallback."

SAY THIS OUT LOUD TWICE

"How does your app call the LLM?"

THE 60-SECOND ANSWER

"Through a provider interface, so the same code works with a hosted OpenAI-compatible API, a local model, or a fake one in tests. Answers stream to the user as server-sent events. Every call has a timeout; I retry only timeouts, connection errors, 429s and 5xx errors, with capped exponential backoff and jitter, and never after text has started streaming. If the main model fails I fall back to a second one. Structured outputs are validated with Pydantic, and every call logs input and output tokens, latency and cost, so I know exactly what a question costs."

Quick self-check

1 Name 4 errors you retry and 4 you never retry.

Retry: timeout, connection error, 429, 503. Never: 400, 401, 403, 404.

2 Why no retry after text was streamed?

The user would see the answer start again or duplicated.

3 Attempt 2, base 0.5 s: what is the wait window?

0 to 2 seconds, random inside it.

4 When does the fallback answer?

Only on LLMUnavailable, and only before any text was sent.

5 Cost of 1,000 input + 500 output tokens at $0.50 / $1.50 per million?

$0.0005 + $0.00075 = $0.00125.

6 What does GET /usage read?

logs/cost.jsonl, grouped by UTC date.

ONE PAGE TO REVISE

Session 5 cheat sheet

Stream

SSE: `data: {json}` + blank line

Last event: `event: done` with usage + cost

Same total time, faster first word

Timeout

`httpx.Timeout(30, connect=5)`

30 s = longest silence, not total time

No timeout = a hung server

Retry

Yes: timeout, connection, 429, 500/502/503/504

Never: 400, 401, 403, 404

Never after text was streamed

Backoff + jitter

random(0, min(8, 0.5 × $2^{attempt}$))

Windows 0.5, 1, 2, 4, 8 s

3 retries: at most 3.5 s of waiting

Fallback

`LLM_FALLBACK_MODEL` in .env

Only on LLMUnavailable, only before text

Log `model` + `fallback_used`

Cost

in × price_in + out × price_out, per million

1,200 + 300 tokens ≈ $0.00105 (example)

Biggest lever: fewer, better chunks

Log + /usage

One JSON line per call in logs/cost.jsonl

`GET /usage` : calls, tokens, cost per day

Estimated usage is never a bill

Commands

`ruff check . && pytest -q` → 14 passed

`curl -N -X POST …/chat`

`curl …/usage`

MEMORY HOOK

"Some Rains Fall, People Look": Stream, Retry, Fall back, Price, Log.

FIVE THINGS TO REMEMBER

Recap and homework

- ✓ The route talks to an interface; providers and the fallback are configuration.
- ✓ Streaming with SSE cuts time to first token; the total time stays the same.
- ✓ Timeouts always; retry only transient errors, with capped backoff and jitter; never after streaming.
- ✓ A fallback model answers only when the main one is unavailable, never to hide a 401.
- ✓ Log tokens, latency, cost and the model for every call; GET /usage totals them per day.

Homework GEN5 · docs/COST.md (due before Thu 29 Oct GenAI)

1. Push today's code: `ruff check .` and `pytest -q` green (14 passed).
2. Ask 10 questions from **your own** `docs/QUESTIONS.md` through `/chat` (the real model if you have a key; the fake one if not, and say so).
3. Write `docs/COST.md`: the model and prices you used (with the pricing page link), average input and output tokens per question from `GET /usage`, cost per question and per 1,000 questions, and one way you would cut it for your product.
4. Commit and push. Wednesday afternoon is the assessment, so finish this tonight.

> Today your project stopped being a demo that works when nothing goes wrong. It now **streams, survives failures and knows its own price**.

TONIGHT
push: 14 tests green

BEFORE THU 29 OCT
docs/COST.md committed

NEXT GENAI · THU 29 OCT
Docker and Docker Compose

MISSED GENAI 4? TYPE THESE FIRST

Monday's files · part 1

These are the GenAI 4 files as today's code expects them. If your Monday version differs slightly, keep yours; just make sure the names match.

APP/CONFIG.PY (THE THREE FALLBACK LINES ARE FROM TODAY)

```python
from functools import lru_cache

from pydantic_settings import BaseSettings, SettingsConfigDict


class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    app_name: str = "examate-rag"
    environment: str = "local"  # local | ci | production

    llm_provider: str = "fake"  # fake | openai_compat
    llm_base_url: str = "http://localhost:11434/v1"
    llm_api_key: str = ""
    llm_model: str = "llama3.1:8b"

    llm_timeout_seconds: float = 30.0
    llm_max_retries: int = 3
    llm_backoff_base_seconds: float = 0.5

    # Added for Session 5: a second model that answers when the main one is unavailable.
    # Empty LLM_FALLBACK_MODEL = no fallback. Base URL and key default to the main ones.
    llm_fallback_model: str = ""
    llm_fallback_base_url: str = ""
    llm_fallback_api_key: str = ""

    price_input_per_million: float = 0.0
    price_output_per_million: float = 0.0
    cost_log_path: str = "logs/cost.jsonl"


@lru_cache
def get_settings() -> Settings:
    return Settings()
```

APP/TOKENS.PY

```python
"""Token estimation when the provider does not report usage (about 4 characters per token)."""


def estimate_tokens(text: str) -> int:
    return max(1, round(len(text) / 4))
```

REQUEST MODELS

Monday's files · part 2

The limits matter: at most 50 messages and 20,000 characters per message. Unbounded input is how one user burns your whole token budget.

APP/SCHEMAS.PY

```python
from pydantic import BaseModel, Field


class Message(BaseModel):
    role: str = Field(pattern="^(system|user|assistant)$")
    content: str = Field(min_length=1, max_length=20_000)


class ChatRequest(BaseModel):
    messages: list[Message] = Field(min_length=1, max_length=50)
    temperature: float = Field(default=0.2, ge=0.0, le=2.0)


class Usage(BaseModel):
    input_tokens: int = 0
    output_tokens: int = 0
    estimated: bool = False  # True when the provider did not report usage and we estimated it


class StructuredAnswer(BaseModel):
    """Example of a structured output schema validated with Pydantic (Session 4)."""

    answer: str
    confidence: float = Field(ge=0.0, le=1.0)
    sources: list[str] = []
```

PAUSE AND PREDICT

A request arrives with `"role": "hacker"`. What does FastAPI return?

Answer: 422. The pattern on `role` rejects it before your code runs; `test_chat_rejects_bad_input` checks this.

THE INTERFACE AND THE FAKE PROVIDER

Monday's files · part 3

Create the folder `app/llm/` with an empty `__init__.py`.

APP/LLM/BASE.PY (LLMUNAVAILABLE AND THE TWO STREAMRESULT LINES ARE FROM TODAY)

```
"""The provider abstraction: the rest of the app only knows this interface."""
from collections.abc import AsyncIterator
from typing import Protocol

from app.schemas import Message, Usage


class StreamResult:
    """Collects usage while a stream is consumed (filled in by the provider)."""

    def __init__(self) -> None:
        self.usage = Usage()
        self.model = ""  # added for Session 5: which model actually answered
        self.fallback_used = False  # added for Session 5


class LLMProvider(Protocol):
    name: str
    model: str

    def stream_chat(
        self, messages: list[Message], temperature: float, result: StreamResult
    ) -> AsyncIterator[str]:
        """Yield text pieces as they arrive; set result.usage before finishing."""
        ...


class LLMError(Exception):
    """Raised when the provider fails after all retries."""


class LLMUnavailable(LLMError):
    """Added for Session 5: the provider was down, slow or rate-limited after every retry.
    Only this error switches to the fallback model; a bad key (401) or wrong model (404)
    is a configuration bug that the fallback must not hide."""
```

NO NETWORK, NO KEY

Monday's files · part 4

APP/LLM/FAKE.PY

```python
"""A deterministic provider for tests, CI and offline demos. No network, no key."""
import asyncio
from collections.abc import AsyncIterator

from app.llm.base import StreamResult
from app.schemas import Message, Usage
from app.tokens import estimate_tokens


class FakeProvider:
    name = "fake"

    def __init__(self, model: str = "fake-model", delay_seconds: float = 0.0) -> None:
        self.model = model
        self.delay_seconds = delay_seconds

    async def stream_chat(
        self, messages: list[Message], temperature: float, result: StreamResult
    ) -> AsyncIterator[str]:
        question = messages[-1].content
        reply = f"You asked: {question}. This is a fake answer for testing."
        for word in reply.split(" "):
            if self.delay_seconds:
                await asyncio.sleep(self.delay_seconds)
            yield word + " "
        result.usage = Usage(
            input_tokens=sum(estimate_tokens(m.content) for m in messages),
            output_tokens=estimate_tokens(reply),
            estimated=True,
        )
```

MEMORY HOOK

The fake provider is your stunt double. It takes every hit in tests and CI so the real model never has to.

WHERE THE FACTS COME FROM

Sources and honest notes

#	PAGE	WHAT IT SUPPORTS	URL
S1	OpenRouter docs: API rate limits	Free model variants: 20 requests/minute; 50/day under 10 credits	`openrouter.ai/docs/api/reference/limits`
S2	Groq docs: Rate limits	Per-model free-plan limits (RPM, RPD, TPM, TPD)	`console.groq.com/docs/rate-limits`
S3	Google AI for Developers: OpenAI compatibility	Gemini OpenAI-compatible base URL; support still in beta	`ai.google.dev/gemini-api/docs/openai`

Honest notes

- [ ] Free-tier numbers come from the providers' own pages at the time the GenAI 02 doc was written and change often: re-check before you rely on them.
- [ ] Prices in the cost example are hypothetical, chosen to make the arithmetic clear.
- [ ] Code marked "added for this session" (fallback, LLMUnavailable, fallback_used, GET /usage and their tests) is new. All code in these notes was run with the fake provider and mock transports: ruff passes and pytest shows 14 passed. No real hosted provider was called while testing.
- [ ] The time-budget suggestion for fallbacks (LLM_MAX_RETRIES=1) is our recommendation, not a provider rule.
- [ ] All interview questions are PATTERN-ONLY: no company is claimed.